S Y S T E M A R C H I T E C T U R E R E V I E W

گزارش کارشناسی معماری سیستم

ارزیابی فنی پلتفرم NEXUS//SIGNALS

موضوع: تحلیل ریشه‌ای خاموشی داشبورد و بازخوانی معماری

محرر: واحد مهندسی NEXUS-LIVE

تاریخ: ۱۷ شهریور ۱۴۰۵ — ۸ سپتامبر ۲۰۲۶

طبقه‌بندی: محرمانه — استفاده داخلی

فهرست مطالب

خلاصه مدیریتی..... 1
شرح رخداد و اهداف گزارش..... 2
دامنه بررسی، شواهد و روش تشخیص..... 2
معماری کلی سیستم NEXUS-LIVE..... 2
لایه ارائه و رابط کاربری..... 3
لایه سرویس و رابط‌های برنامه‌نویسی..... 3
موتور معاملاتی و سرویس‌های جانبی..... 3
لایه داده و عملیات..... 4
تحلیل ریشه‌ای خاموشی داشبورد..... 4
خط زمانی رخداد..... 4
زنجیره علل ریشه‌ای..... 4
رله تلگرام DORMANT یافته فرعی: وضعیت..... 5
اقدامات اصلاحی انجام‌شده و راستی‌آزمایی..... 5
ارزیابی ریسک‌ها و آسیب‌پذیری‌های معماری..... 6
توصیه‌ها و نقشه راه پایدارسازی..... 6

توجه: این فهرست با کدهای فیلد تولید شده است. پس از هر ویرایش، برای به‌روزرسانی شماره صفحات، روی فهرست راست‌کلیک کرده و گزینه Update Field را انتخاب کنید.

خلاصه مدیریتی

در تاریخ ۸ سپتامبر ۲۰۲۶ برابر با ۱۷ شهریور ۱۴۰۵، داشبورد وب پلتفرم NEXUS//SIGNALS برای اپراتور قابل مشاهده نبود. بررسی اولیه نشان داد سرویس اصلی برنامه (Next.js روی پورت 3000) پاسخ نمی‌دهد، در حالی که دو سرویس جانبی جریان قیمت (پورت 3003) و پایش نقدشوندگی (پورت 3010) سالم بودند. تشخیص به‌صورت لایه‌به‌لایه از سطح شبکه تا هسته سیستم انجام شد و ریشه رخداد در همان روز شناسایی و برطرف گردید. این گزارش شرح کامل زنجیره رخداد، مستندسازی معماری سیستم و نقشه راه پایدارسازی را در بر می‌گیرد.

علت ریشه‌ای، توقف پروسه next-server به دست OOM Killer کرنل لینوکس بود. این پروسه پیش از توقف حدود ۲٫۳ گیگابایت حافظه فیزیکی مصرف کرده بود، در حالی که میزبان تنها ۴ گیگابایت حافظه و بدون هیچ فضای swap است. نکته قابل‌توجه آنکه مصرف واقعی از سقف ۷۶۸ مگابایتی پیکربندی‌شده عبور کرده بود؛ بررسی نشان می‌دهد این سقف تنها هیپ V8 را مهار می‌کند و حافظه native کامپایلر Turbopack در حالت توسعه از آن پیروی نمی‌کند. به بیان دیگر، سیستم حفاظتی موجود برای سناریوی رخ‌داده طراحی نشده بود.

سرویس با اسکریپت استاندارد scripts/dev-up.sh بازیابی شد و سلامت آن در سه لایه مستقل راستی‌آزمایی گردید: پاسخ HTTP سالم با ۵۷٬۹۷۷ بایت HTML کامل، بازگشت داده زنده از API پالس، و رندر کامل صفحه در مرورگر تست بدون هیچ خطای کنسول. یک یافته فرعی نیز ثبت شد: پس از هر راه‌اندازی مجدد، رله تلگرام تا پیام بعدی اپراتور در وضعیت DORMANT می‌ماند، زیرا شناسه گفت‌وگو در فایل env ذخیره نشده است. مهم‌ترین توصیه‌های گزارش عبارت‌اند از: اتصال مکانیزم خودترمیمی به job نگه‌دارنده، مهاجرت از حالت dev به بیلد production، افزودن swap و پایش فعال حافظه.

شرح رخداد و اهداف گزارش

در نوبت عصر ۸ سپتامبر ۲۰۲۶، اپراتور هنگام بازکردن صفحه پیش‌نمایش وب، داشبورد را دریافت نکرد. هم‌زمان job نگه‌دارنده sandbox (شناسه 354312) در همان بازه زمانی، پاسخ پورت اصلی را ناموفق با کد 000 گزارش کرد؛ در همان فراخوانی، سرویس نقدشوندگی کد 200 و سرویس قیمت کد 400 موردانتظار خود را برگرداندند. الگوی «زنده بودن سرویس‌های فرعی و خاموشی سرویس اصلی» نخستین سرنخ فنی بود و مسیر تشخیص را از ابتدا به سمت پروسه برنامه اصلی هدایت کرد.

اهداف این گزارش سه مقصد روشن را دنبال می‌کند. نخست، پاسخ مستند و مستند به پرسش «چرا داشبورد نمایش داده نشد» با استناد به شواهد خام سیستم؛ دوم، توصیف ساختار معماری فعلی به‌گونه‌ای که نقاط قوت و ضعف آن برای تصمیم‌گیری قابل قضاوت باشد؛ و سوم، ارائه توصیه‌های اجرایی اولویت‌بندی‌شده برای جلوگیری از تکرار رخداد. مخاطب این سند اپراتور سیستم است و از ورود به جزئیات استراتژی معاملاتی و پارامترهای سیگنال‌دهی عمداً پرهیز شده است تا تمرکز بر مسئله عملیاتی باقی بماند.

دامنه بررسی، شواهد و روش تشخیص

شواهد این بررسی از پنج منبع مستقل گردآوری شد: لاگ توسعه برنامه (dev.log)، خروجی‌های job نگه‌دارنده در دو نوبت متوالی، جدول سوکت‌های گوش‌دهنده سیستم (ss)، فهرست پروسه‌های زنده (ps)، و بافر پیام‌های هسته (dmesg) همراه با خروجی وضعیت حافظه (free). جمع این منابع امکان تفکیک دقیق زمان رخداد،

مکانیسم توقف و وضعیت اجزای پیرامون را فراهم کرد و هیچ‌یک از نتیجه‌گیری‌های این گزارش به یک منبع واحد متکی نیست.

روش تشخیص، پیمایش لایه‌به‌لایه از علائم به علت بود: ابتدا لایه شبکه و رفتار نامتقارن پورت‌ها بررسی شد، سپس لایه پروسه که نبود فرایند next در آن آشکار شد، و سرانجام لایه هسته ردپای قطعی را در قالب رکورد OOM kill نشان داد. برای اطمینان از رفع کامل، پس از بازیابی، صفحه با مرورگر headless باز شد و کنسول و خطاهای صفحه پایش گردید. بازه بررسی از آخرین راه‌اندازی موفق سرویس تا لحظه بازیابی است؛ یک محدودیت جزئی آن است که مهر زمانی رکورد OOM در dmesg بر حسب uptime کرنل ثبت می‌شود و نگاشت آن به ساعت دیواری تقریبی است.

معماری کلی سیستم NEXUS-LIVE

پلتفرم NEXUS-LIVE یک ترمینال فرمان معاملاتی خودران است که هسته آن یک اپلیکیشن Next.js نسخه ۱۶ با App Router و TypeScript و Tailwind 4 و اجزای shadcn/ui است. در کنار اپلیکیشن اصلی، دو سرویس سبک مبتنی بر Bun و یک پایگاه داده SQLite (از طریق ORM پرازیما با حالت WAL) فعالیت می‌کنند. جدول ۱ نمای کلی اجزای در حال اجرا و وضعیت هر یک در لحظه رخداد را نشان می‌دهد.

جدول ۱ — اجزای در حال اجرا و وضعیت آن‌ها در لحظه رخداد

مؤلفه	پورت / مسیر	نقش در سیستم	وضعیت حین رخداد
اپلیکیشن اصلی Next.js (حالت dev با Turbopack)	3000	رابط داشبورد، تمام APIها و میزبانی موتور معاملاتی	خاموش — توقف با OOM
سرویس قیمت price-ws (Bun)	3003	جریان قیمت زنده WebSocket برای تیکر و نمودارها	سالم
سرویس نقدشوندگی liq-service (Bun)	3010	پایش رخدادهای نقدشوندگی بازار	سالم
پایگاه داده SQLite + Prisma	db/custom.db	ذخیره سیگنال‌ها، رویدادها، بازار و تنظیمات	سالم (حالت WAL)
رله تلگرام @Neo_agentaibot	خارجی	آینه هشدارها و ارسال گزارش روزانه با تصویر	فعال؛ پس از ری‌استارت DORMANT

چیدمان کلی در شش لایه قابل توصیف است: لایه ارائه، لایه سرویس، موتور معاملاتی، لایه داده، سرویس‌های جانبی و لایه عملیات. تفکیک مسئولیت‌ها میان این لایه‌ها در بخش‌های بعدی تشریح می‌شود و همین تفکیک است که باعث شد دامنه آسیب رخداد فعلی به یک مؤلفه محدود بماند.

لایه ارائه و رابط کاربری

رابط کاربری در قالب یک صفحه واحد با پنج نما ارائه می‌شود: داشبورد، سیگنال‌ها، تحلیل‌ها، ترمینال رویدادها و هسته هوش مصنوعی؛ جابه‌جایی میان نماها توسط یک مسیریاب وضعیت مبتنی بر Zustand انجام می‌گیرد. داده‌های زنده از دو کانال تأمین می‌شوند: polling پنج‌ثانیه‌ای روی نقطه پایانی /api/pulse با TanStack Query، و جریان قیمت WebSocket از سرویس پورت 3003. نمودارها با Recharts و حرکت‌های رابط با Framer Motion ساخته شده‌اند و مجموعه‌ای از اجزای مشترک مانند BentoCard و NeonBadge و Sparkline میان همه نماها بازاستفاده می‌شوند؛ همین انسجام باعث شده صفحه داشبورد پس از بازگشت سرویس، بدون هیچ تغییری به کار عادی برگردد.

لایه سرویس و رابط‌های برنامه‌نویسی

حدود ۲۵ مسیر API زیر همان اپلیکیشن Next.js سرویس می‌شوند و ستون فقرات آن‌ها نقطه پایانی pulse است که در یک فراخوانی اسنپ‌شات کامل بازار، سیگنال‌های فعال، رویدادها و آمار عملکرد را برمی‌گرداند. مسیرهای مدیریت سیگنال شامل فهرست‌گیری، ایجاد و بستن دستی است؛ مسیرهای هوش مصنوعی (گفت‌وگو، تحلیل، ایجنت، فرماندار، کالیبراسیون، یادگیری عمیق و حافظه) به‌صورت فقط-سروری به SDK هوش مصنوعی متصل‌اند؛ و مسیرهای تخصصی مانند vol-onset و vol-cause و flow/matrix و liquidations و backtest و گزارش روزانه، تحلیل‌های اختصاصی را ارائه می‌کنند. مسیر گزارش روزانه دارای گارد ضدهرز سخت‌گیرانه است که حداکثر یک ارسال در روز را حتی با پرچم force مجاز می‌داند و همین مکانیزم در تست دستی امروز نیز به‌درستی از ارسال تکراری جلوگیری کرد.

موتور معاملاتی و سرویس‌های جانبی

موتور خودران در قالب تابع tick با دوره حدوداً ۳٫۵ تا ۴ ثانیه کار می‌کند: قیمت‌ها را به‌روزرسانی می‌کند، سیگنال‌ها را در چرخه حیات کامل (باز شدن، عبور از اهداف TP1 تا TP3، توقف ضرر یا انقضای TTL) مدیریت می‌کند و رویدادهای سیستم را برای ترمینال تولید می‌سازد. لایه حاکمیت ریسک شامل governor و ttl-gate و calibration است که نرخ تولید سیگنال و انضباط انقضا را کنترل می‌کنند. دو سرویس جانبی روی Bun اجرا می‌شوند: price-ws جریان قیمت را broadcast می‌کند و liq-service نقدشوندگی بازار را پایش می‌کند. جداسازی این دو به‌صورت پروسه مستقل از نظر مهندسی حافظه تصمیم درستی است، چون خرابی آن‌ها به اپلیکیشن اصلی سرریز نمی‌شود؛ همان‌طور که در این رخداد هر دو تا آخرین لحظه سالم ماندند.

لایه داده و عملیات

پایگاه داده SQLite با حالت WAL و پارامتر busy_timeout پنج‌ثانیه‌ای کار می‌کند و تمام وضعیت کسب‌وکار سیستم شامل سیگنال‌ها، رویدادها، تاریخچه قیمت و تنظیمات را نگه می‌دارد. پشتیبانی‌گیری هر سه ساعت توسط job شماره 352556 از طریق اسکریپت backup.sh انجام می‌شود و بسته فشرده به‌همراه مانیفست در مسیر backups نگهداری می‌شود. چهار job زمان‌بندی در زنجیره عملیات فعال‌اند: نگه‌دارنده sandbox (354312)، پشتیبانی‌گیری سه‌ساعته (352556)، بازبینی توسعه وب (353472) و گزارش روزانه تلگرام (364970). اسکریپت dev-up.sh راه‌انداز idempotent سیستم است که سرویس را با سقف هیپ ۷۶۸ مگابایتی بالا می‌آورد، و حلقه auto-discover رله تلگرام با پایش دوره‌ای getUpdates شناسه گفت‌وگوی اپراتور را کشف می‌کند.

تحلیل ریشه‌ای خاموشی داشبورد

خط زمانی رخداد

جدول ۲ توالی رویدادها را از آخرین عملکرد سالم تا بازیابی کامل نشان می‌دهد. زمان‌ها به وقت تهران و برای لحظات میانی تقریبی است؛ نشانه‌های ستون سوم مستقیماً از شواهد خام استخراج شده‌اند.

جدول ۲ — خط زمانی رخداد بر اساس شواهد گردآوری‌شده

زمان (تهران)	رخداد	نشانه / منبع
۰۰:۲۷	ارسال موفق گزارش روزانه به تلگرام با تصویر DAILY.jpg	پاسخ route و رکورد lastSentAt
۱۷:۳۱	job نگه‌دارنده سالم: app:200	خروجی کرون 354312
۱۷:۴۵	نگه‌دارنده: app:000؛ تلاش مجدد پس از ۳ ثانیه: 200 در ۵٫۵ ثانیه	کامپایل سرد /api/pulse در dev.log
۱۷:۵۰~	پاسخ صفحه اصلی صفر بایت و سپس رد فوری اتصال	curl: code 000 در ۰٫۱۴ میلی‌ثانیه
۱۷:۵۲~	بدون هیچ listener روی پورت 3000 و بدون پروسه next	خروجی ss و ps
۱۷:۵۵~	رکورد هسته: توقف پروسه next-server به دلیل کمبود حافظه	dmesg: OOM kill با RSS حدود ۲٫۳GB
۱۷:۵۸~	بازیابی با dev-up.sh و راستی‌آزمایی سه‌لایه	app:200 و رندر کامل در مرورگر تست

زنجیره علل ریشه‌ای

رکورد هسته نتیجه را قطعی می‌کند: پیام توقف پروسه 17003 با نام next-server و مصرف anon-rss برابر ۲٬۳۱۴٬۱۱۲ کیلوبایت (حدود ۲٫۳ گیگابایت) و حافظه مجازی حدود ۲۶ گیگابایت صادر شده است. میزبان ۴٬۰۴۱ مگابایت حافظه کل دارد و swap آن صفر است؛ بنابراین هنگامی که RSS پروسه اصلی رشد کرد، کرنل ناگزیر بزرگ‌ترین مصرف‌کننده را حذف کرد و این یعنی همان پروسه‌ای که داشبورد را سرویس می‌کرد. از آن پس هر اتصال به پورت 3000 بلافاصله رد شد و مرورگر کاربر صفحه‌ای خالی یا خطای اتصال دید؛ این دقیقاً همان چیزی است که اپراتور به‌صورت «داشبورد نمایش داده نمی‌شود» تجربه کرد.

پرسش بعدی این است که چرا مصرف از سقف ۷۶۸ مگابایتی پیکربندی‌شده عبور کرد. پاسخ در معماری ابزارهاست: متغیر NODE_OPTIONS با محدودیت max-old-space-size فقط هیپ جاوااسکریپت موتور V8 را مهار می‌کند، در حالی که کامپایلر Turbopack در Next.js 16 یک مؤلفه native است و کش‌ها و حافظه آن خارج از این سقف رشد می‌کند. در لاگ، نشانه‌های همین رفتار دیده می‌شود: کامپایل‌های سرد مکرر مسیرها پس از هر دوره بی‌کاری، مانند /api/pulse با ۳٫۷ ثانیه کامپایل و ۱٬۷۸۰ میلی‌ثانیه رندر، یعنی کش‌ها به‌طور مکرر تخلیه و بازساخته می‌شوند. کارکرد بیش از دو روز بی‌وقفه در حالت dev این الگوی مصرف را به یک رشد تجمعی و در نهایت غیرقابل‌تحمل برای میزبان تبدیل کرد.

و چرا یک توقف پروسه به قطعی کامل تبدیل شد؟ چون در زنجیره عملیاتی فعلی هیچ مؤلفه‌ای مسئول بازگرداندن پروسه اصلی نیست: job نگه‌دارنده فقط ping می‌کند و نتیجه را گزارش می‌دهد؛ اسکریپت dev-up.sh قابلیت راه‌اندازی مجدد دارد اما هیچ فراخوان خودکاری ندارد؛ و اسکریپت watchdog نیز در زنجیره زمان‌بندی فعال نیست. در نتیجه نخستین رخداد OOM مستقیماً به یک قطعی قابل‌مشاهده برای کاربر نهایی تبدیل شد. شایان‌ذکر است که پنجره آسیب‌پذیری پیش‌تر نیز دیده شده بود: در نوبت ۱۷:۴۵ پاسخ pulse پس از تلاش مجدد ۵٫۵ ثانیه طول کشید که برای کاربر همان تجربه «صفحه‌ای که باز نمی‌شود» است، هرچند سرور رسماً زنده بود.

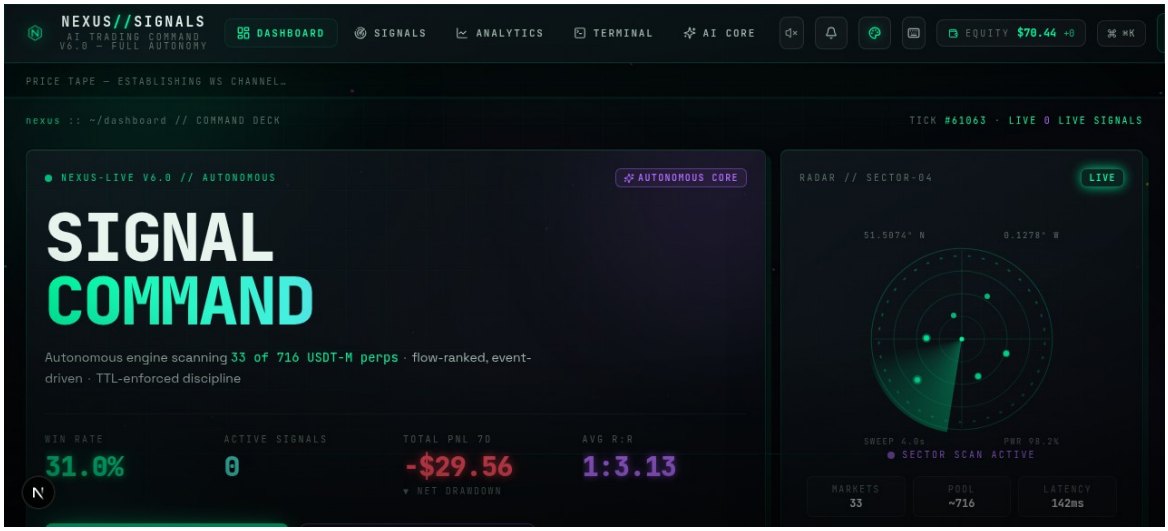
یافته فرعی: وضعیت DORMANT رله تلگرام

پس از راه‌اندازی مجدد سرویس، لاگ پیام خاموشی موقت رله را ثبت کرد: تلگرام تا دریافت پیام بعدی اپراتور در حالت DORMANT می‌ماند. بررسی فایل env نشان می‌دهد فقط کلید TELEGRAM_BOT_TOKEN تعریف شده و شناسه گفت‌وگو در زمان اجرا و از طریق حلقه کشف خودکار به دست می‌آید؛ بنابراین پس از هر ری‌استارت، زنجیره هشدار تا نخستین پیام بعدی اپراتور کور است. گزارش روزانه امشب پیش از رخداد با موفقیت ارسال شده بود و گارد ضدهرز نیز در تست دستی به‌درستی عمل کرد، اما این یافته نشان می‌دهد پایداری زنجیره اعلان‌ها به یک state در حافظه وابسته است که با هر چرخه عمر سرویس از دست می‌رود.

اقدامات اصلاحی انجام‌شده و راستی‌آزمایی

بازیابی سرویس با اجرای اسکریپت استاندارد scripts/dev-up.sh انجام شد. این اسکریپت نخست سلامت را می‌سنجد، در صورت عدم پاسخ، پروسه‌های مزاحم پورت را پاک می‌کند، سرور را در نشست مستقل و با سقف هیپ ۷۶۸ مگابایتی بالا می‌آورد و تا دریافت پاسخ 200 از نقطه پایانی pulse حداکثر شصت ثانیه منتظر می‌ماند. خروجی اجرا app:200 بود و سرویس در کمتر از یک دقیقه به چرخه کاری بازگشت.

راستی‌آزمایی در سه لایه مستقل انجام شد تا از رفع کامل اطمینان حاصل شود. در لایه HTTP، صفحه اصلی کد 200 را با ۵۷٬۹۷۷ بایت HTML کامل برگرداند؛ در حالت خرابی همین درخواست صفر بایت یا رد اتصال بود. در لایه داده، نقطه پایانی pulse اسنپ‌شات زنده بازار را با قیمت بیت‌کوین حدود ۷۸٬۷۸۵ دلار بازگرداند که به معنی سلامت موتور و پایگاه داده است. در لایه رندر، مرورگر تست صفحه را با عنوان صحیح بارگذاری کرد، صفر خطای کنسول و صفر خطای صفحه ثبت شد و تصویر شکل ۱ از وضعیت نهایی داشبورد گرفته شد. سرویس‌های پورت 3003 و 3010 در تمام طول رخداد بدون وقفه فعالیت کردند و هیچ داده‌ای از دست نرفت.



شکل ۱ — داشبورد NEXUS//SIGNALS بلافاصله پس از بازیابی؛ رندر کامل در مرورگر تست بدون خطای کنسول

ارزیابی ریسک‌ها و آسیب‌پذیری‌های معماری

نخستین جمع‌بندی مهم آن است که این رخداد یک نقص منطقی در موتور معاملاتی یا لایه داده نبود: سلامت داده‌ها حفظ شد، سرویس‌های فرعی پابرجا ماندند و زنجیره گزارش روزانه نیز پیش از رخداد عملکرد درست داشت. آسیب تمام‌وکمال در لایه عملیات رخ داد؛ جایی که سه ضعف ساختاری هم‌زمان حضور داشتند: تک‌پروسه‌ای بودن اپلیکیشن اصلی بدون هیچ سوپروایزری، بودجه حافظه تنگ میزبان چهار گیگابایتی بدون swap که باید سه پروسه مستقل، کامپایلر و مرورگرهای تست را هم‌زمان جا دهد، و اجرای طولانی‌مدت حالت توسعه با Turbopack در نقش محیط ماندگار ۲۴ ساعته.

بدون اقدام اصلاحی، احتمال تکرار رخداد بالا ارزیابی می‌شود؛ زیرا عامل رشد حافظه یعنی کش‌های کامپایلر و رفتار تجمعی حالت dev پابرجاست و تنها مهارکننده فعلی یعنی سقف هیپ V8 همان‌طور که بررسی شد ناقص است. اثر کسب‌وکاری خاموشی، توقف مشاهده سیگنال‌ها و هشدارها در داشبورد برای اپراتور بود؛ خوشبختانه داده‌ها پشتیبانی‌گیری‌شده‌اند و ریسک از دست رفتن اطلاعات پایین تلقی می‌شود، اما تداوم تجربه کاربر به خطر افتاده است. در کنار این، یافته فرعی رله تلگرام نشان داد زنجیره اعلان‌های حیاتی نیز در برابر چرخه عمر سرویس مقاوم نیست و در سناریوی بدتر که هم‌زمانی خاموشی با یک رخداد بازار باشد، اپراتور ممکن است بدون هیچ هشداری باقی بماند.

توصیه‌ها و نقشه راه پایدارسازی

شش اقدام زیر بر اساس نسبت اثر بر دسترس‌پذیری به هزینه اجرا اولویت‌بندی شده‌اند. دو مورد با اولویت P0 را می‌توان در همان نشست عملیاتی جاری اعمال کرد و به‌تنهایی از تکرار سناریوی امروز جلوگیری می‌کنند؛ موارد P1 در افق یک هفته و مورد P2 به‌عنوان نگه‌داشت دوره‌ای پیشنهاد می‌شوند.

جدول ۳ — اقدامات پیشنهادی اولویت‌بندی‌شده

اولویت	اقدام	اثر مورد انتظار	نقطه اجرا
P0	اتصال dev-up.sh به job نگه‌دارنده با الگوی: پینگ؛ در نبود 200 اجرای بازیابی	سقف هر قطعی به کمتر از ۲ دقیقه	کرون 354312
P0	افزودن سنجش RSS پروسه به pulse و هشدار تلگرامی در آستانه ۱٫۲ گیگابایت	پیش‌بینی و پیش‌گیری از OOM	مسیر pulse / notify
P1	مهاجرت به بیلد production با next build و next start	حذف کامپایل runtime و کاهش شدید حافظه	استقرار
P1	افزودن ۲ گیگابایت swap به میزبان	ضربه‌گیر OOM در جهش‌های حافظه	سیستم‌عامل
P1	ماندگارسازی شناسه گفت‌وگوی تلگرام در پایگاه داده و فعال‌سازی خودکار پس از بوت	زنجیره هشدار بدون فاصله کور	notify / auto-discover
P2	بازچرخشی برنامه‌ریزی‌شده سرویس در کم‌بارترین ساعت شبانه‌روز	مهار رشد تجمعی حافظه تا مهاجرت کامل	زنجیره کرون

درباره اجرای دو مورد P0: تغییر job نگه‌دارنده به الگوی «پینگ و در صورت نیاز بازیابی» عملاً منطق آماده dev-up.sh را به یک حلقه خودترمیم تبدیل می‌کند و تنها یک خط فرمان بیشتر نمی‌خواهد؛ سنجش حافظه نیز با خواندن RSS پروسه از procfs و الحاق آن به پاسخ pulse امکان‌پذیر است و آستانه پیشنهادی، هشدار در ۱٫۲ گیگابایت و راه‌اندازی مجدد کنترل‌شده در ۱٫۶ گیگابایت است. درباره مهاجرت به production، بیلد نهایی حافظه کامپایل را یک‌بار و در زمان ساخت مصرف می‌کند و زمان اجرا را سبک و بدون کامپایل می‌کند؛ تا پیش از آن، بازچرخشی شبانه ریسک تجمع حافظه را مهار می‌کند و افزودن swap نیز بدون هیچ تغییری در کد، حاشیه امن کرنل را ایجاد می‌کند.

جمع‌بندی نهایی: معماری NEXUS-LIVE از نظر تفکیک مسئولیت‌ها میان اپلیکیشن، سرویس‌های جانبی و داده، و از نظر الگوهای مهندسی به‌کاررفته مانند WAL، پشتیبانی‌گیری دوره‌ای، گارد ضدهرز اعلان‌ها و حاکمیت ریسک چندلایه، سالم و ماژولار ارزیابی می‌شود. رخداد بررسی‌شده محصول یک شکاف عملیاتی قابل‌ترمیم است، نه نقص طراحی. با اجرای کامل موارد P0 و P1، انتظار می‌رود دسترس‌پذیری داشبورد روی همان زیرساخت فعلی به سطحی بالای ۹۹٫۹ درصد برسد و پنجره‌های صفحه خالی برای کاربر نهایی به‌طور کامل حذف شود.